Genetic Algorithm vs. Random Search for Configuring Production RAG Retrieval

Problem Identification

Retrieval-Augmented Generation(RAG) reduces hallucination in question-answering systems by using relevant documents at query time and giving them to a language model as context, instead of relying on the model's trained weights [1]. Businesses increasingly deploy RAG over an internal knowledge base, where wrong answers bring more risk. In the HW 1 literature survey on production RAG, I found that benchmark performance often does not carry over to deployment. A separate review of 63 studies found that less than 15% addressed real integration to production [2].

The most consistent finding in my survey is that no single RAG configuration is best. Index choice depends on workload: IVFFLat builds 20x faster than HNSW while HNSW reaches slightly higher peak recall when fine tuned [4], and IVFFlat can outperform HNSW under low-selectivity metadata filtering [3]. Chunking strategies showed similar trade off results where some configurations may have given better relevance but at a 4x cost in embedding cost[5]. Barker et al. found that while tuning full RAG pipelines an optimal configuration on one task did not generalize well to new tasks [7]. This indicates configurations have to be tuned per task or model. This leads to a large increase in compute cost as each configuration requires new embeddings and testing.

Hyperparameter optimization is an ongoing area of research. Grid search, a traditional form of hyperparameter search, scales exponentially with the number of hyperparameters. Bergstra and Bengio worked on a random search method that is more efficient in higher dimensions as only a few hyperparameters mattered on any given task. They recommended it as a baseline for adaptive methods to try and beat [8]. Genetic algorithms (GAs) are one potential adaptive method. Lee et al. encoded a CNN's structure, activation function, and optimizer as binary chromosomes and evolved it with rank-based selection, uniform crossover, and mutation factors [9]. Ozcelik and Efe's survey of evolutionary neural architecture search (NAS) shows GAs are becoming more popular in designing the structure of neural networks [10].

The cost of searching larger and larger spaces for model architecture and hyper parameters continues to increase exponentially as model size and amount of data used to train them also increases. GAs have been seen as a potential way to help minimize computational cost when it comes to determining NN architecture structure. The same GA methods used for NN architecture can also be used to help find RAG configurations. Automated tuning methods for RAG do exist. Methods such as AutoRAG, using a greedy search over RAG modules [11], and AutoRAG-HP, framing tuning as a hierarchical multi-armed bandit problem [12], but none of the sources I found used GA for RAG configuration.

This project aims to connect these concepts together. I want to use a GA implemented with the python LEAP (Library for Evolutionary Algorithms) package from ORNL to search RAG configurations on a fixed subspace (determined by compute power and size of sources). I will aim to compare it against the random and grid search methods that have previously been explored. The goal is to quantify how much search cost can be cut and what quality tradeoffs occur.

Research Question Development

As previously stated, RAG has no universal best configuration, so tuning for the problem is necessary. GAs are a potential tuning method for finding a high quality configuration. Previous work leaves the comparison of GAs with random search methods for RAG unanswered. The goal of this project is to attempt to bridge that gap.

Research Question: Does a GA find RAG configurations closer to the best achievable than random search at an equal evaluation budget.

Hypothesis: At an equal evaluation budget, a GAs best found configuration will have a smaller gap to the best achievable score than a random search.

Methods Outline

This is a quantitative, computational experimental research study. Every candidate RAG configuration receives a repeatable retrieval quality score, so the study is a controlled benchmark. The independent variable is the search method (genetic algorithm or random search). The dependent variable is the gap between the best configuration from the full grid search and the best found by each of the two search methods. The evaluation budget, search space, question set, and fitness function will be held constant across all methods.

I will use a fixed document set with questions and labeled relevant passages. The current frontrunner for the dataset is the WixQA dataset, a well known dataset for evaluating and training RAG systems[13]. The search space will be limited to a small area in order to minimize computation in the grid search, roughly 500-1,000 configurations in total. The genome will include information about chunk size, chunk overlap, retrieval type, index type, reranker toggle, and top-k. These are all different hyperparameters used to tune RAG. For this study only the model's retrieval will be evaluated and not the generated answers themselves. I will evaluate every configuration once using nDCG@k (Normalized Discounted Cumulative Gain at K), a metric that evaluates the quality of ranked lists by measuring how well the top k items are ordered by relevance, as the fitness score. These values will then be placed into a lookup table to help minimize performance and fitness computations when testing the methods across different random seeds.

This study will use Python with LEAP and other relevant packages as needed. Because every configuration will be pre-scored, the GA and random search methods will become lookups in the table instead of having to recalculate each time. This is designed to allow for hundreds of independent runs per method at dramatically reduced costs. The three conditions being compared in this study are:

Condition	Role
Exhaustive Grid Search of full space	Reference for lookup table / best possible
Random Search	Baseline (Bergstra and Bengio [8])
Genetic Algorithm	Method under test

The “budget” for each method will be counted as the number of unique configurations evaluated, with revisited configurations counted as once, and the same rule applying to both Random Search and the Genetic Algorithm. I will test multiple budgets making sure to keep the budgets small enough to not capture the entire search grid. GA settings will be fixed before looking at comparison results. These settings will be determined from my previous lab experience working with LEAP and other genetic algorithms. Each method will be run several times with different random seeds. The number of random seeds tested will be determined after seeing the time it takes to check the random seed on both methods. For each run, the main score for comparison will be the value of the best possible score minus the best score the method found. The analysis will include convergence curves (best so fat gap against evaluations), the mean and median gap at each budget with confidence intervals, and a Mann-Whitney U test comparing the two methods.

As for the ethical considerations, one of the goals for this project is to find which method achieves better scores by consuming as little compute cost as possible in an effort to reduce energy and carbon cost, however in doing the study we will have to run an exhaustive grid search in order to best compare the other methods. Meaning in order to complete the study we will have to use the objectively worst method in terms of energy and carbon cost in order to find improvements.

Justification

A controlled experiment suits this question because the gap to the best achievable configuration, is a measurable and repeatable value to achieve. Random search is the standard baseline that an adaptive method can try to beat, and an exhaustive grid search provides ground truths and allows for later comparison between the other methods to be significantly cheaper due to the lookup tables. This allows me to run several different random seeds and report variance without drowning in time to compute. By using a budget for evaluation I am able to keep comparison fair between both the GA and Random Search methods.

References

[1] P. Lewis *et al.*, “Retrieval-augmented generation for knowledge-intensive NLP tasks,” in *Proc. 34th Conf. Neural Inf. Process. Syst. (NeurIPS)*, Vancouver, Canada, 2020.

[2] E. Karakurt and A. Akbulut, “Retrieval-augmented generation (RAG) and large language models (LLMs) for enterprise knowledge management and document automation: A systematic literature review,” *Appl. Sci.*, vol. 16, no. 1, p. 368, 2026, doi: 10.3390/app16010368.

[3] A. Amanbayev, B. Tsan, T. Dang, and F. Rusu, “Filtered approximate nearest neighbor search in vector databases: System design and performance analysis,” arXiv:2602.11443, 2026.

[4] W. Yang, T. Li, G. Fang, and H. Wei, “PASE: PostgreSQL ultra-high-dimensional approximate nearest neighbor search extension,” in *Proc. ACM SIGMOD Int. Conf. Management of Data*, Portland, OR, USA, 2020, pp. 2241–2253, doi: 10.1145/3318464.3386131.

[5] C. Merola and J. Singh, “Reconstructing context: Evaluating advanced chunking strategies for retrieval-augmented generation,” arXiv:2504.19754, 2025.

[6] R. Ruparel and S. Bussari, “Survey and benchmarking of retrieval methods for enterprise retrieval-augmented generation,” in *Proc. Int. Conf. Computer and Applications (ICCA)*, 2025, doi: 10.1109/ICCA66035.2025.11430920.

[7] M. Barker, A. Bell, E. Thomas, J. Carr, T. Andrews, and U. Bhatt, “Faster, cheaper, better: Multi-objective hyperparameter optimization for LLM and RAG systems,” in *ICLR 2025 Workshop on Foundation Models in the Wild*, 2025, arXiv:2502.18635.

[8] J. Bergstra and Y. Bengio, “Random search for hyper-parameter optimization,” *J. Mach. Learn. Res.*, vol. 13, pp. 281–305, 2012.

[9] S. Lee, J. Kim, H. Kang, D.-Y. Kang, and J. Park, “Genetic algorithm based deep learning neural network structure and hyperparameter optimization,” *Appl. Sci.*, vol. 11, no. 2, p. 744, 2021, doi: 10.3390/app11020744.

[10] F. N. Özçelik and M. Ö. Efe, “Evolutionary neural architecture search: A survey,” *Turk. J. Elec. Eng. Comp. Sci.*, vol. 34, no. 4, pp. 507–541, 2026, doi: 10.55730/1300-0632.4189.

[11] D. Kim, B. Kim, D. Han, and M. Eibich, “AutoRAG: Automated framework for optimization of retrieval augmented generation pipeline,” arXiv:2410.20878, 2024.

[12] J. Fu *et al.*, “AutoRAG-HP: Automatic online hyper-parameter tuning for retrieval-augmented generation,” arXiv:2406.19251, 2024.

[13] D. Cohen *et al.*, “WixQA: A multi-dataset benchmark for enterprise retrieval-augmented generation,” arXiv:2505.08643, 2025.

AI Disclosure:
I used Claude Sonnet 5.5 to help with IEEE citations using articles from the literature review and additional sources I added for the proposal. I checked the doi and arXiv’s when available to check it’s work.